Predicting `cp.async` Pipeline Effectiveness from Memory-Subsystem Parameters: A Cross-GPU Study

Xiangchen Song Chenxin Geng Yuyang Feng Tian Xia

EECS 570 – Parallel Computer Architecture
University of Michigan

Milestone 1 March 18, 2026

Problem & Motivation

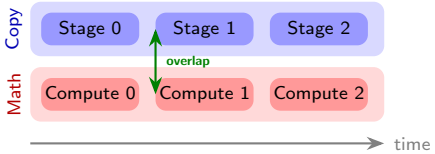
Background:

- NVIDIA `cp.async` (Ampere+) enables asynchronous global→shared memory copies, overlapping with computation
- CUTLASS, FlashAttention-3, Triton all rely on multi-stage `cp.async` pipelines; pipeline depth S is a key tuning knob

The Problem:

- Practitioners tune S on *one* GPU and assume it transfers
- But pipeline effectiveness depends on **L2 capacity**, **memory BW/latency**, and **per-SM shared memory** — which vary *dramatically* across GPUs
- No existing autotuner uses architecture parameters to prune the search space

Multi-stage cp.async Pipeline



Research Question

Does optimal S^* and its speedup diverge across GPUs with different memory subsystems? Can a parameter-free model predict this?

Three Falsifiable Hypotheses

H1 – L2-capacity driven: Speedup inflects when working set $W_{\text{conc}} > C_{\text{L2}}$.

Test: L40S (112 MB L2) should see minimal benefit; A40 (8 MB) should see largest benefit at large N.

H2 – Ridge-point driven: Low ridge-point GPUs benefit most from pipelining.

Test: Speedup ordering A100-MIG (R=7.9) > H100 (R=20) > A40 (R=53.7).

H3 – Latency–occupancy tension: Optimal depth S^* balances DRAM latency hiding vs. SMEM-induced occupancy loss.

Test: $S^*(N)$ non-monotonic on A40 (high latency + small SMEM); flat on H100.

In practice, a **mixture** is likely — the predictive model incorporates all three.

Experimental Setup: Hardware

	V100	A40	A100 MIG	L40S	H100
Arch	Volta	Ampere	Ampere	Ada	Hopper
SMs	80	84	42	142	132
`cp.async`	—	✓	✓	✓	✓
Memory	HBM2	GDDR6	HBM2e	GDDR6	HBM3
Peak BW	900	696	968	864	3350 GB/s
L2 (nominal)	6 MB	6 MB	20 MiB	96 MB	50 MB
L2 (measured)	—	8 MB	TBD	112 MB	28 MB
SMEM/SM	96 KB	100 KB	164 KB	100 KB	228 KB
FP32 peak	15.7	37.4	7.6	45.7	67 TF
Ridge	17.4	**53.7**	**7.9**	52.9	20.0
Role	Boundary	Core contrast		L2 extreme	Core anchor

Design rationale:

- **A40 vs H100**: max memory-subsystem gap (8.3× L2, 4.8× BW)
- **A100 MIG**: lowest ridge (7.9) → H2 predicts most benefit; tests MIG
- **L40S**: largest L2 (96 MB nominal, 112 MB measured) → tests "L2 absorbs all" extreme
- **V100**: no `cp.async` → boundary condition (1.0×)

Key finding: Measured L2 ≠ nominal L2. H100 nominal 50 MB → effective 28 MB (L2 slicing); L40S nominal 96 MB → effective 112 MB.

Experimental Setup: Kernels & Methodology

Two complementary kernel families:

ID	GPUs	Description
V0-GEMM	All	Naive global load, no tiling
V1-GEMM	All	Shared-mem tiling, sync load
V3-GEMM	A40+	`cp.async` pipeline, $S \in \{2, 3, 4\}$
V1-Stencil	All	1D stencil, sync shared-mem
V3-Stencil	A40+	`cp.async` pipelined, $S \in \{2, 3, 4\}$
Ref	All	cuBLAS SGEMM (ceiling)

Same V3 source code runs on A40, A100-MIG, H100 — **no GPU-specific branches**.

Why these two kernels?

- **GEMM**: compute-bound (AI $\gg R$) but still has per-tile memory stalls
- **Stencil**: memory-bound (AI $\approx$ 5 FLOP/B), near/below ridge point
- If predictor works for both $\rightarrow$ captures general memory-subsystem properties

Workload sweep:

- GEMM: $N \in \{256, 512, \ldots, 8192\}$
- Tile sizes: $T \in \{16, 32, 64, 128\}$
- Stencil: $L \in \{2^{16}, 2^{18}, \ldots, 2^{26}\}$

Measurement:

- `cudaEvent_t`, 3 warmup + 10 timed, median
- Correctness: cuBLAS (GEMM) / CPU (stencil)

Predictive Model (Parameter-Free)

Idea: predict `cp.async` speedup from architecture constants alone (no fitted parameters).

Three inputs per GPU (from pointer-chase microbenchmark):

- Effective L2 capacity C_{L2}^{eff}
- L2 latency L_{L2}, DRAM latency L_{DRAM}

Key modeling choices:

- **280-cycle floor** on per-stage compute — sync + async overhead dominates small tiles
- α=0.15 — occupancy loss is highly sub-linear

Tools: CMake + SLURM, Nsight Compute, pointer-chasing microbenchmark

Core Formula

$$h_{\text{hit}} = \min\left(1,\ C_{L2}^{\text{eff}} \,/\, W_{\text{conc}}\right) \qquad \text{L2 hit fraction}$$

$$L_{\text{eff}} = h_{\text{hit}} \cdot L_{L2} + (1 - h_{\text{hit}}) \cdot L_{\text{DRAM}} \qquad \text{effective memory latency (cycles)}$$

$$T_{\text{compute}} = \max\left(T_{\text{compute}}^{\text{raw}},\ 280\right) \qquad \text{per-stage compute window (cycles)}$$

Predicted Speedup

$$\widehat{S}_{\text{pred}} = \underbrace{\frac{\max\left(T_{\text{compute}},\ L_{\text{eff}}\right)}{\max\left(T_{\text{compute}},\ L_{\text{eff}} \,/\, S_{\text{stage}}\right)}}_{\text{overlap benefit}} \times \underbrace{\left(\frac{\text{Occ}(S_{\text{stage}})}{\text{Occ}(1)}\right)^{0.15}}_{\text{occupancy penalty}}$$

C_{L2}^{eff} = effective L2 capacity
W_{conc} = concurrent working set
$T_{\text{compute}}^{\text{raw}}$ = pure FLOP time per stage
L_{L2}, L_{DRAM} = L2 / DRAM latency
S_{stage} = pipeline depth (2,3,4
$\text{Occ}(S)$ = blocks per SM at dep

All inputs = arch constants + microbenchmark values. *Zero* parameters fitted from kernel data.

Results & Key Findings

Model Accuracy (MAPE):

GPU	GEMM	Stencil
A40	10.9%	3.7%
L40S	14.1%	10.6%
H100	9.9%	5.5%
Overall	**11.6%**	**6.6%**

Measured L2 $\neq$ Nominal:

GPU	Nominal	Effective
A40	6 MB	8 MB
L40S	96 MB	112 MB
H100	50 MB	**28 MB**

H100 only 44% of nominal (L2 slice locality).

Hypothesis Status:

H1 **Supported.** $W_{\text{conc}}/C_{\text{L2}}$ is primary driver. L40S (huge L2) $\rightarrow$ minimal benefit; A40 (small L2) $\rightarrow$ largest benefit.

H2 **Partial.** Ridge point alone insufficient — must combine with L2/latency.

H3 **Supported.** A40 tile=64 stage=3: occupancy 3$\rightarrow$1 blocks/SM causes measurable slowdown.

Predicted Optimal Stage:

GPU	Workload	Best S^*	Speedup
V100	All	1	1.00×
A40	All	1	1.00×
L40S	GEMM (tile=16)	2	1.01×
L40S	Stencil	2	1.05×
H100	GEMM (tile=16)	2–4	1.03×
H100	Stencil	2	1.03×

Key insight: Occupancy cost almost always outweighs latency hiding $\rightarrow$ S^*=1 dominates.

Bottom line: ~10% MAPE across 4 architectures with zero fitted parameters.

Next Steps & Timeline

Week	Lead	Milestone
green!10 1–2	A+C	✓ Build system, V0/V1 baselines, pointer-chasing microbench
green!10 3–4	B	✓ V3-GEMM/Stencil impl, initial A40 curves
green!10 5–6	B+D	✓ 5-GPU comparison (V100/A40/A100/L40S/H100), model refinement, **Milestone 1**
7–8	C+D	Nsight anomaly deep-dives, ★Triton pre-filter, A100-MIG data
9–10	All	Pipeline benefit maps, final figures, report + poster

Completed In progress

Remaining work (Weeks 7–10):

- Complete A100-MIG experiments (pending cluster queue)
- Nsight Compute deep-dives on top-5 MAPE outliers
- ★ Triton pre-filter: measure autotune time reduction and false-negative rate
- Pipeline benefit heatmaps (AI $\times$ $W_{\text{conc}}/C_{\text{L2}}$)
- Leave-one-out cross-validation across 5 GPUs
- Final report (6-page, conference format)
- Poster ($30'' \times 40''$)

Key risk:

A100-MIG L2 behavior may differ from full A100; if pointer-chase fails on MIG, fall back to nominal L2 with sensitivity analysis.